2.2、图像算数&逻辑运算

[OpenCV: Arithmetic Operations on Images]
https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html

这是 OpenCV 中最基础也最常用的操作之一，学会它之后，你就能做很多酷炫的事情，比如：

- 两张图融合、渐变叠加
- 把 logo 无缝贴到照片上（不带矩形白底）
- 简单的图像增强
- 用掩码（mask）做区域替换

1、图像加法

```
import cv2 as cv
import numpy as np

# 方法1：用 cv.add() —— 推荐（饱和运算）
cv.add(img1, img2)

# 方法2：直接用 numpy 加法 —— 不推荐用于图像
img1 + img2   # 会模256运算，容易出现颜色翻转（黑变白、白变黑）
```

非常重要的区别（很多人踩坑的地方）：

```
x = np.uint8([250])
y = np.uint8([10])

print(cv.add(x, y))     # [[255]]   ← 饱和到255，符合人眼预期
print(x + y)            # [4]       ← 260 % 256 = 4，颜色严重错误！
```

结论：图像像素加法 → 永远用 cv.add() 或 cv.addWeighted()

2. 图像混合 / 加权融合（Image Blending）—最常用技巧

公式（记住这个就懂了本质）：

```
结果 = α × 第一张图 + β × 第二张图 + γ
```

通常 γ = 0，α + β = 1

```
img1 = cv.imread('ml.png')          # 底图
img2 = cv.imread('opencv-logo.png') # 要叠加的图

# 0.7 是底图权重，0.3 是logo权重
dst = cv.addWeighted(img1, 0.7, img2, 0.3, 0)

cv.imshow('blended', dst)
cv.waitKey(0)
cv.destroyAllWindows()
```

3. 位运算（Bitwise Operations）—— 贴 logo / 抠图神器

这是教程里最有“魔法感”的部分：把 OpenCV logo 贴到 Messi 照片左上角，而且不带矩形白边。完整经典代码：

```
import cv2 as cv
import numpy as np

# 1. 读图
img1 = cv.imread('messi5.jpg')           # 背景图（梅西）
img2 = cv.imread('opencv-logo-white.png') # logo（最好是白底透明或纯黑白）

# 2. 确定 logo 大小，并取出背景对应区域（ROI）
rows, cols, channels = img2.shape
roi = img1[0:rows, 0:cols]               # 注意：img1 必须够大

# 3. 把 logo 转灰度 → 二值化 → 得到掩码（mask）
img2gray = cv.cvtColor(img2, cv.COLOR_BGR2GRAY)
ret, mask = cv.threshold(img2gray, 10, 255, cv.THRESH_BINARY)  # 10是阈值，根据
图调整
```

```
# 4. 反掩码（logo以外的区域）
mask_inv = cv.bitwise_not(mask)

# 5. 把背景的 logo 区域“抠黑”（只保留非logo部分）
img1_bg = cv.bitwise_and(roi, roi, mask=mask_inv)

# 6. 只保留 logo 的彩色部分
img2_fg = cv.bitwise_and(img2, img2, mask=mask)

# 7. 两者相加 → 得到合成结果
dst = cv.add(img1_bg, img2_fg)

# 8. 贴回原图
img1[0:rows, 0:cols] = dst

cv.imshow('result', img1)
cv.waitKey(0)
cv.destroyAllWindows()
```

核心思想（一句话记住）用二值掩码（mask）控制“哪些像素要保留、哪些像素要清零”，然后通过位运算实现“抠掉背景 + 只保留前景”的分离，最后再把前景贴回去。

```
# 假设 img2 是 logo（有背景色，比如白色或黑色）
# img1 是背景大图（梅西照片）

rows, cols = img2.shape[:2]
roi = img1[0:rows, 0:cols]          # 取出要替换的区域

gray = cv.cvtColor(img2, cv.COLOR_BGR2GRAY)
```

```
_, mask = cv.threshold(gray, 10, 255, cv.THRESH_BINARY)   # logo 主体 ≈ 255，背景 ≈ 0

mask_inv = cv.bitwise_not(mask)     # 反转：主体 ≈ 0，背景 ≈ 255

# 步骤关键 ↓↓↓
img1_bg   = cv.bitwise_and(roi, roi, mask=mask_inv)   # 只保留 roi 中“非 logo 形状”的部分（把 logo 区域变黑）
img2_fg   = cv.bitwise_and(img2, img2, mask=mask)     # 只保留 logo 的彩色部分（背景变黑）
dst       = cv.add(img1_bg, img2_fg)                  # 黑+彩色 = 无缝合成
img1[0:rows, 0:cols] = dst                            # 贴回去
```

3.1. cv.threshold(gray, thresh, maxval, type) → 制造二值掩码

最常用组合：cv.THRESH_BINARY

参数	常见值	含义
src	灰度图	单通道 8bit 图像
thresh	5~30	阈值（要调！取决于 logo 背景亮度）
maxval	255	超过阈值的像素设成这个值（通常 255）
type	THRESH_BINARY	> thresh → maxval，否则 → 0

结果：

- logo 主体（较暗/较亮部分）变成 255（白色）
- 背景变成 0（黑色）
  这张 mask 就是“我想保留的区域”。

3.2. cv.bitwise_not(mask) → mask_inv

作用：把 255 变成 0，把 0 变成 255
物理意义：得到“我要清零 / 保护的区域”现在我们有两张互补的掩码：

- mask ： 白色 = logo 形状（前景要保留）
- mask_inv ： 白色 = logo 以外（背景要保留）

3.3. cv.bitwise_and(src1, src2, dst, mask=xxx) ← 核心中的核心

这是最容易搞混的地方，我们分两种情况讲清楚。

情况A：没有 mask 参数（或 mask=None）

```
cv.bitwise_and(a, b)   # 等价于 每个通道： dst = a & b
```

像素级按位与，常用于：

- 两张图取交集
- 颜色通道做过滤

情况B：有 mask 参数（最常用在贴图）

- 官方文档原话（非常关键）：
- 当提供了 mask 时，只在 mask 非零（白的）的像素位置才执行 src1 & src2 的运算；
- mask 为 0 的位置，dst 保持不变（但因为我们通常传入 dst 是空的或同 src1，所以表现为“保持 src1”或“被清零”）。
- 但在实际贴 logo 代码里，我们最常见的两种写法：

```
# 写法1：把 logo 区域“抠黑”（最常见）
img1_bg = cv.bitwise_and(roi, roi, mask=mask_inv)

# 写法2：只提取 logo 前景彩色部分
img2_fg = cv.bitwise_and(img2, img2, mask=mask)
```

为什么 src1 和 src2 都传同一个图？因为我们想要的效果是：

- 只保留“mask 非零”位置的原始像素值
- mask 为零的位置 → 全部变成 0（黑色）
  一句话理解：

```
cv.bitwise_and(img, img, mask=mask)   ≈   “只保留 mask 白色的区域，其它地方全黑”
```

所以：

目的	代码写法	mask 应该传什么	结果含义
保留 logo 形状的彩色	bitwise_and(img2, img2, mask=mask)	mask（前景白）	logo 彩色部分保留，背景全黑
把 roi 的 logo 区域清黑	bitwise_and(roi, roi, mask=mask_inv)	mask_inv（背景白）	roi 非 logo 部分保留，logo 区域变黑

3.4. cv.add(img1_bg, img2_fg)

因为两个图的非重叠区域都是黑的（0），所以：

- 0 + 彩色 = 彩色
- 背景 + 0 = 背景
- 不存在重叠的彩色冲突
  → 完美无缝合成快速对照表（建议打印贴墙上）

想达到的效果	mask 应该是什么样子	典型代码写法	结果图像特点
只保留形状内的彩色	形状内 255，外 0	bitwise_and(img, img, mask=mask)	形状彩色，其余全黑
只保留形状外的原图	形状外 255，形状内 0	bitwise_and(roi, roi, mask=mask_inv)	原图 + 形状区域变黑
把某区域变黑（抠掉）	要抠掉的地方是 0	bitwise_and(img, img, mask=要保留的地方)	要抠的地方黑掉
把某区域强制变白/亮	—	bitwise_or + 常数，或直接赋值	—

3.5.图像的与运算

你原来的认知（“与运算应该是 0 和 1 之间的布尔运算”）其实在逻辑层面是正确的，但在图像处理（和计算机底层）中，位运算的“与”是对每个 bit（比特）逐位进行的，而不是把整个像素值当作一个 0/1 的布尔值。

1. OpenCV 的图像（绝大多数情况）是 uint8 类型 → 每个通道取值范围 0 ~ 255
2. 掩码（mask）也是 uint8 类型，常用两种值：
   - 0 （黑色，不通过）
   - 255 （白色，通过）
3. bitwise_and、bitwise_or 等位运算 是对每个字节的每一位（bit）逐位做运算
   所以掩码的 255 和 0 在二进制里是这样的：

```
255 → 1111 1111     （8 个 bit 全是 1）
  0  → 0000 0000     （8 个 bit 全是 0）
```

逐位与运算的真实过程（举例）假设某个像素的原始值是 10（十进制），我们用掩码来“与”它：

```
原始像素值：    0000 1010    （十进制 10）
掩码是 255：    1111 1111
----------------------------
逐位与的结果：  0000 1010   → 仍然是 10
```

```
掩码是    0:    0000 0000
----------------------------
逐位与的结果:  0000 0000   → 变成 0
```

再来一个更典型的例子（掩码是 255 或 0 的情况最常见）：

原始像素值 (十进制)	二进制	掩码值	掩码二进制	结果 (十进制)	结果二进制	含义
180	1011 0100	255	1111 1111	180	1011 0100	完全保留
180	1011 0100	0	0000 0000	0	0000 0000	完全清零（变黑）